

Задание «Пропускная способность компьютерной сети»

- Загружать решения в виде ZIP архива в проект Smart LMS под названием HW1 (дедлайн указан в SmartLMS).
- В архиве должна быть папка, которая включает в себя отчет¹ (DOCX/PDF формат), проект Java и файл XLSX с графиками (см. ниже).
- Название ZIP архива должно совпадать с Вашей фамилией.

Мотивация. Если Ваше программное обеспечение попадает в новую компьютерную сеть, вначале следует выполнить измерения характеристик новой коммуникационной среды.

В первую очередь, важно знать скорость обмена данными по сети именно для Вашего конкретного приложения и бизнес-протокола (вместо general-purpose tools like iperf/ping). Дополнительно мотивация иллюстрируется на лекции 20 января 2026 (см. также слайды).

На семинарах уже должны были быть освещены базовые сетевые средства языка Java, работа в среде IntelliJ IDEA. В том числе, предполагалось изучение класса InetAddress, сокетов клиента и сервера, замеров времени с помощью System.currentTimeMillis(), упаковка в JAR, размещение и запуск JAR. Если Вы выполнили задание № 0, хотя оно не оценивается, его решение послужит хорошей основой для проекта по этому заданию.

Для работы над этим заданием Вам могут быть предоставлены виртуальные машины в Облаке (см. ниже). Преподаватели помогут Вам с техническими вопросами, например, подключению к машине в Облаке, работе с JAR файлами, WireShark, определение IP через ifconfig/ipconfig. При этом вопросы организации передачи и программирования этой передачи – **это часть задания**, эту часть Вам необходимо будет выполнить самостоятельно, т.е. самостоятельно продумать и запрограммировать способы обмена данными между клиентом и сервером, также выполнить логику экспериментов, собрать данные и подготовить отчет.

Перед началом работы:

- Пожалуйста, создавайте Maven/Gradle проект.
- Установить Oracle JDK 25 <https://www.oracle.com/java/technologies/downloads/> (является последней LTS версией; используйте именно Oracle JDK 25, а не другие JDK)
- Установить IntelliJ IDEA Community либо Eclipse <https://www.jetbrains.com/idea/download> либо <https://eclipseide.org/>
- Установить WireShark <https://www.wireshark.org/download.html>
- Эти инструменты нужны для локальной разработки и отладки

Загружать в LMS – один архив ZIP

- язык для выполнения работы – Java
- отчет в DOCX/PDF формате (имя файла – Ваша фамилия)
- проект Maven/Gradle (без файлов IntelliJ IDEA, .idea и других файлов)
- файл XLSX с графиками
- скриншоты WireShark, htop/nethogs (демонстрация работы Вашего клиент-серверного приложения)

¹ Примечание: без заполнения ключевых разделов отчета, напр. об использовании удаленных серверов или ИИ, работа может быть оценена с коэффициентом 0.5 (т.е., по умолчанию не были использованы удаленные серверы либо был использован ИИ, шаблон отчета находится в конце этого файла) – в данной работе не измеряется скорость меж-процессного обмена данными между клиентом и сервером, работающими на одной и той же физической машине (localhost), клиент и сервер должны размещаться на разных машинах при сборе статистики (см. ниже)

Содержание задания.

Реализовать клиент-серверное приложение на Java (и сервер и клиент должны быть реализованы на Java). Использовать класс Socket на стороне клиента и класс ServerSocket на стороне сервера.

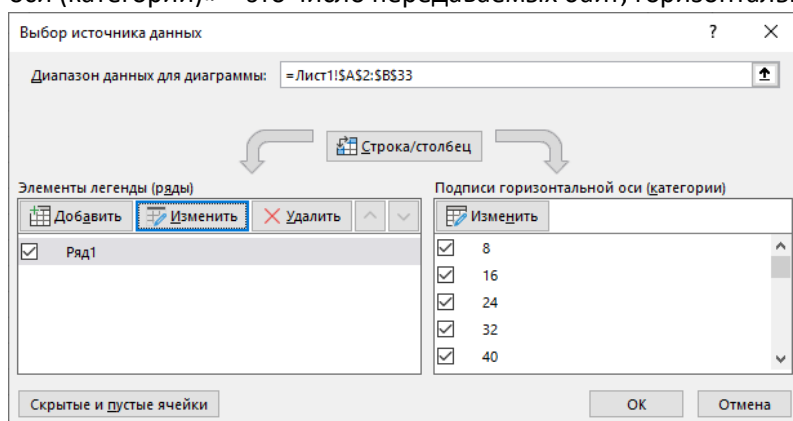
Клиент имеет следующие параметры командной строки: <IP> <port> <N> <M> <Q>. Клиент должен подключаться к серверу по IP адресу <IP> и порту <port>. Клиент должен выполнить M итераций и Q под-итераций отсылки/приема данных. На итерации номер $K = 0, \dots, M-1$ клиент отправляет серверу массив байт длиной $N \cdot K + 8$ несколько раз, а именно Q раз.

Клиент каждый раз при отсылке формирует массив из случайного набора байт (чтобы предотвратить его кэширование либо оптимизацию доставки на каких-либо этапах). Также следует установить флаг TCP_NO_DELAY. Клиент перед очередной отсылкой массива должен дожидаться ответа от сервера. Клиент замеряет время, которое требуется для того, чтобы отправить массив байт и дождаться ответа от сервера. Для итерации K требуемое время рассчитывается усреднением времени за Q под-итераций.

Сервер в ответ должен отправлять строку YYYY.MM.DD HH:MM:SS (текущая дата и время; можно добавить временную зону; можно использовать класс Date и формировать ответ вручную либо DateTimeFormatter).

Клиент не выводит в консоль ответы от сервера. Клиент замеряет для каждого запроса время, которое прошло от вызова метода write до получения ответа от сервера (System.currentTimeMillis, не используйте System.nanoTime). Для каждого значения K клиент рассчитывает среднее время. В конце работы клиент выводит в файл либо на консоль таблицу (число байт итерации K, среднее время). По этим данным необходимо построить график в Excel (горизонтальная ось – число байт итерации K, вертикальная – среднее время). Пожалуйста, интерпретируйте полученный график.

При построении графика, обратите внимание – «Элементы легенды (ряды)» — это время, вертикальная ось: нужно выбрать столбец с измеренным временем, а «Подписи горизонтальной оси (категории)» — это число передаваемых байт, горизонтальная ось.



Выполнить минимум три сценария:

1. Сценарий 1 (Маленькие размеры массивов). Значения: $N = 8$, $M = 5000$, $Q = 25$. Таким образом, на итерации номер 0 клиент отправляет серверу 8 байт, на итерации 1 отправляет 8+8 байт, ..., на итерации $M-1$ отправляет $8 \cdot (M-1) + 8$ байт.
2. Сценарий 2 (Большие размеры массивов). Значения: $N = 1024$, $M = 5000$, $Q = 10$.
3. Сценарий 3 (Ваши значения). Указанные выше значения N , M , Q – не единственно возможная конфигурация, при которой можно проследить интересующие нас закономерности, проведите эксперименты с разными входными параметрами, напр., $N = 1$ либо 8 либо 64 либо 1024 и т.п., $M = 2$ в степени, $Q = 1$ либо 25 и так далее и постройте графики минимум для 3-х разных комбинаций параметров.

Указания.

Обязательно выполнить отслеживание пакетов сервера и клиента в Wireshark (достаточно найти несколько пакетов и понять, что они сгенерированы Вашими клиентом и сервером).

Необходимо использовать на стороне клиента

```
Socket s = new Socket("127.0.0.1", server_port); -- IP надо будет заменить
socket.setTcpNoDelay(true); -- попробовать и с true и с false
s.getOutputStream();
```

на стороне сервера:

```
ServerSocket ss = new ServerSocket(server_port);
Socket s = ss.accept();
s.getInputStream();
```

Развертывание клиента и сервера.

1. Во время разработки и тестирования, сервер и клиент могут располагаться на Вашей рабочей машине (localhost).
2. Во время проведения полномасштабного эксперимента есть несколько вариантов.
3. Вариант 1. Вы можете воспользоваться удаленными виртуальными машинами (VM) нашего курса в Облаке (предоставляется в индивидуальное пользование студенту 1–2 VM).
Внимание: машины будут доступны во время занятия либо консультации (уточняйте время консультации у преподавателя семинаров).
 - a. Используйте VM только для выполнения домашнего задания.
 - b. Используйте предоставленный Вам логин и пароль.
 - c. Пожалуйста, не запускайте на VM посторонние, не относящиеся к заданию программы, чтобы не занимать ресурсы.
 - d. Ваше приложение-сервер не является требовательным к ресурсам, поэтому заранее подготовьте jar файл (командой `java -jar ...`) с Вашим приложением-сервером, скопируйте его на VM и запускайте именно jar файл из командной строки.
 - e. Выделяйте Вашему приложению-серверу не более 100–200 МБ оперативной памяти: use the arguments `-Xms<memory> -Xmx<memory>`. Use M or G after the numbers for indicating Megs and Gigs of bytes respectively. `-Xms` indicates the minimum and `-Xmx` the maximum.
 - f. Ваше клиентское приложение может размещаться либо на Вашей собственной машине, либо на машине в аудитории, либо клиент и сервер могут размещаться в облаке на 2х разных VM, находящихся в разных регионах (географических зонах).
 - g. Для доступа к виртуальным машинам лучше использовать удаленный рабочий стол.
4. Вариант 2. У Вас есть доступ к VDS/VPS серверу, либо виртуальным машинам (VM) в Облаке, другим удаленным сетевым ресурсам. В таком случае Вы можете выполнить работу с их использованием (см. сценарии и рекомендации по размещению клиента и сервера выше).
5. В любом случае/варианте, в отчете Вам следует привести факты, подтверждающие запуск клиента и сервера на разных машинах.
6. Для чистоты эксперимента убедитесь, что машины не загружены посторонними тяжеловесными задачами (напр., используйте утилиты `htop/nethogs` во время работы Вашего приложения)

ПОДГОТОВЬТЕСЬ ЗАРАНЕЕ К ЭКСПЕРИМЕНТАМ, ПРОДУМАЙТЕ КАКИМ ОБРАЗОМ ВЫ БУДЕТЕ СОБИРАТЬ ДАННЫЕ (НАПР., ЗАПИСЫВАТЬ В CSV ФАЙЛ), ЧТОБЫ ВО ВРЕМЯ ПОСЕЩЕНИЯ СЕМИНАРА ЛИБО КОНСУЛЬТАЦИИ ВЫ СМОГЛИ ЭФФЕКТИВНО ИСПОЛЬЗОВАТЬ ВРЕМЯ ЗАНЯТИЯ.

Оценивание данного задания будет происходить следующим образом. Во-первых, код будет анализироваться на корректность и точность выполнения требований этого задания. Далее, будет выполнен анализ полученных графиков и отчета. Затем будет учитываться качество кода (форматирование, обработка corner cases, иные стандартные элементы качества кода).

Шаблон отчета

ОТЧЕТ ПО HW1

ФИО, группа, email

Цель работы: *укажите цель работы своими словами*

Я претендую на оценку до 5 баллов, был использован генеративный ИИ (далее следует информация из декларации согласно https://www.hse.ru/studyspravka/ai_guidelines/)

ЛИБО

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Были использованы такие удаленные машины для клиента или сервера (указать для чего именно, либо и для клиента, и для сервера):

здесь необходимо привести факты, свидетельствующие о том, что клиент и сервер не размещались на одной физической машине, например:

- IP адреса клиента и сервера
- Скриншоты WireShark
- Скриншоты traceroute/tracert
- иные факты

Преподаватель может запросить и другие подтверждения при необходимости, в Ваших интересах привести полную картину экспериментальной установки.

Далее кратко опишите какие собственные значения (Q, M, N) были выбраны и по каким причинам, каким образом были проведены эксперименты (в т.ч. скриншоты htop/nethogs во время работы Вашего приложения), что на Ваш взгляд показалось интересным, странным, либо требующим внимания.

Кратко опишите получившиеся графики (скриншот – описание), 1–3 предложения. В частности, ответьте на вопрос о том, какие закономерности Вы увидели на графиках.